

Pushing the Frontier for Data Agents with Genie

by The Databricks AI Research Team

Genie is Databricks' state-of-the-art data agent designed for answering complex questions about enterprise data consisting of both structured (tables, dashboards, notebooks, etc.) and unstructured (workspace files, Google Drive, Sharepoint etc.) data sources. This blog describes some of the unique challenges faced by data agents and introduces techniques to address them, including using specialized knowledge search, parallel thinking, and Multi-LLM designs. From our experiments on an internal benchmark of real-world data analysis tasks, we observe that these techniques can significantly improve the overall accuracy of Genie over a leading coding agent (from 32% to over 90%) while also significantly reducing the costs and latency.

Optimizing Genie with Specialized Knowledge Search, Parallel Thinking, and Multi-LLM

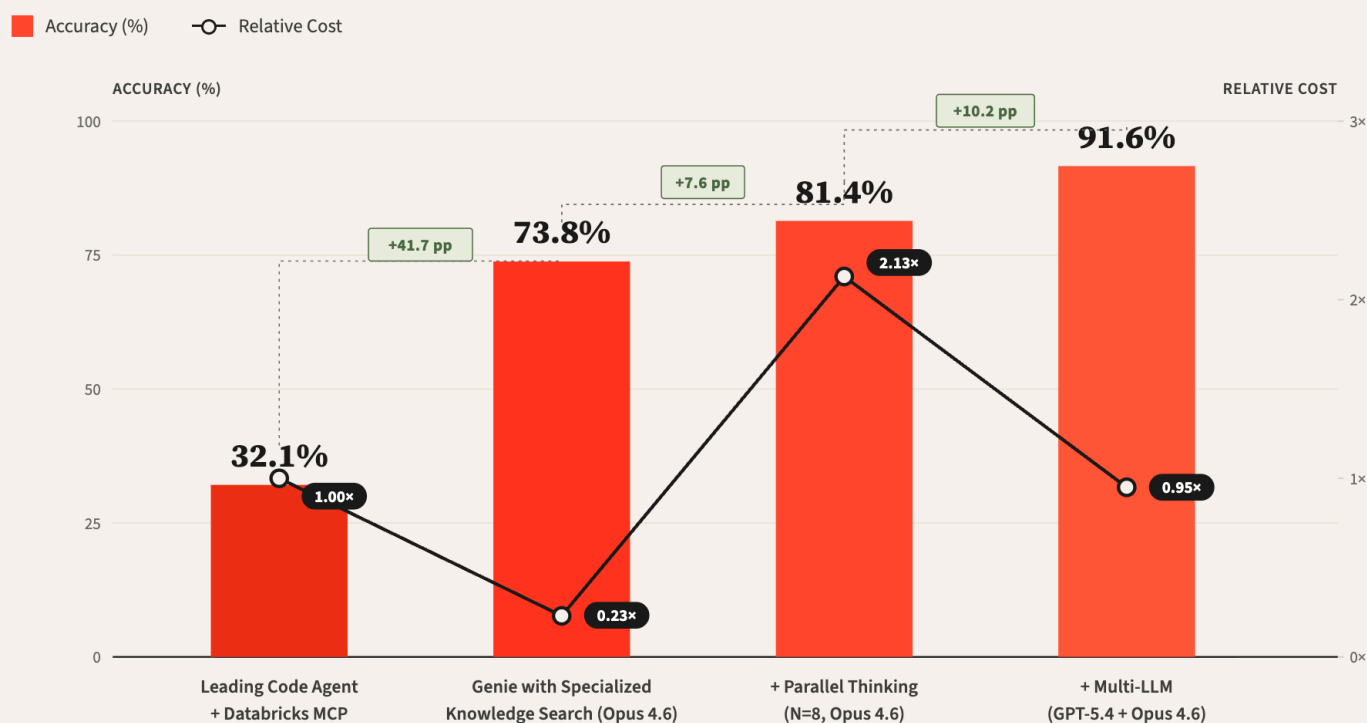


Figure 1: A plot of Genie experiments using different techniques such as specialized knowledge search, parallel thinking, and a Multi-LLM design with optimized prompts.

Key Challenges for Data Agents

Coding agents have shown that a powerful LLM can do incredible things autonomously when equipped with tools that help it understand the code context. While coding agents operate effectively in static, deterministic environments like a disk's file system, *data* agents introduce an entirely new paradigm. Data agents work within a dynamic, constantly evolving data lakehouse that encompasses a wealth of semantic context across hundreds of thousands of tables, notebooks, dashboards, and documents.

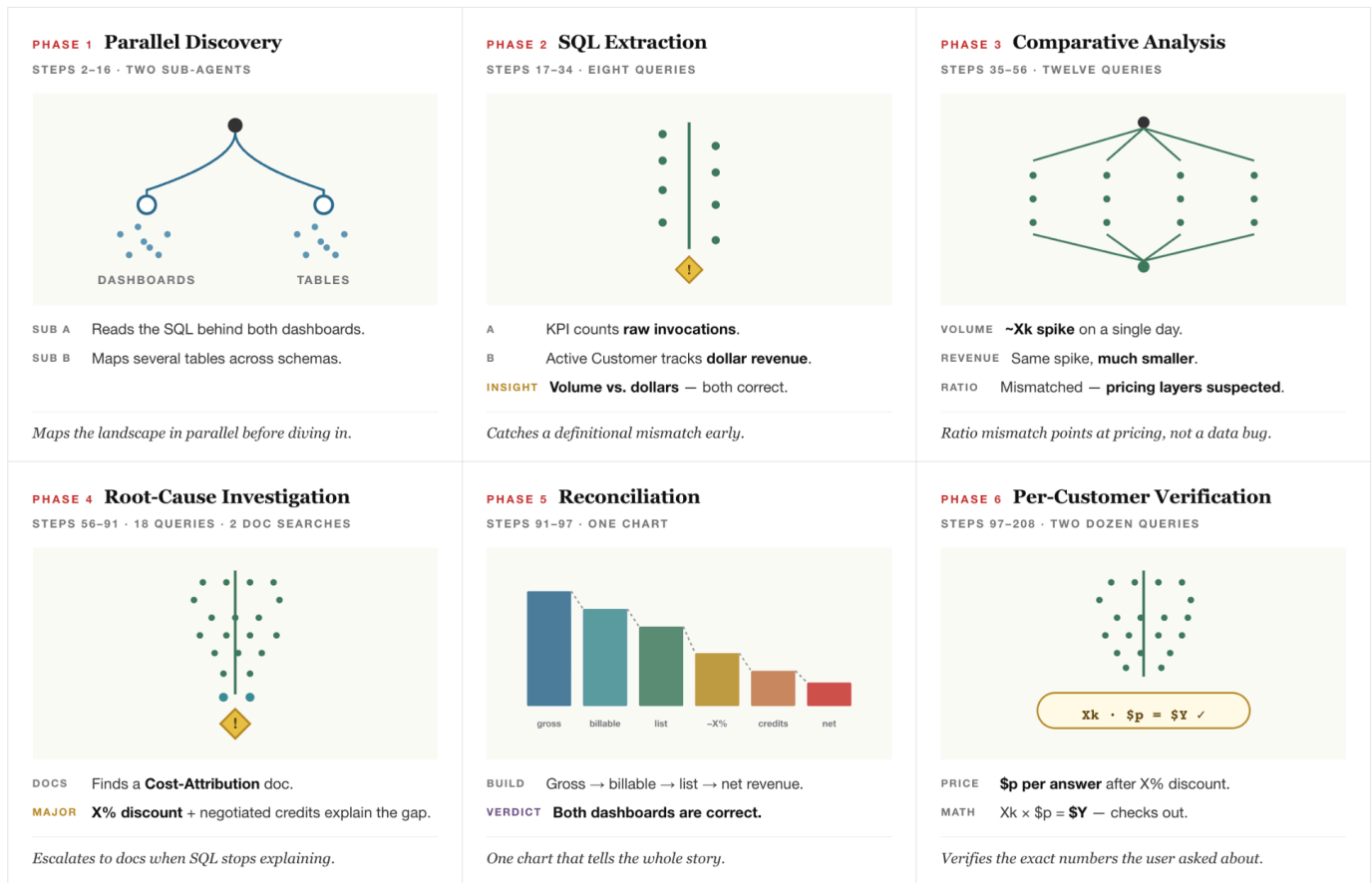
For example, consider a real (anonymized) query asked by an internal user in Figure 2: the user notices that two enterprise dashboards reporting the same product's revenue show contradictory spikes on different dates and asks the agent to explain why. This reasonable question is deceptively hard because no single data source contains the answer and resolving the question requires cross-system discovery across tables, internal documents, and dashboards, and reasoning about how multi-day reports are set up. Additionally, it requires the agent to dig into enterprise pricing details to find contract rates. Finally, it requires the agent to have an ability to automatically correct itself when intermediate calculations reveal incorrect initial assumptions. The figure shows how the agent is able to successfully solve the task by proceeding in different phases: (1) parallel multi-agent data discovery, (2) data investigation, (3) self-correction loop, and (4) verification.

Compared to Coding Agents, Data Agents have three key unique challenges:

- **Scale of Data Discovery:** Finding the right data sources to answer the user query is one of the biggest challenges with enterprise customers having millions of structured and unstructured sources (like tables, dashboards, and documents), a scale that breaks conventional search methods.
- **Determining "Source of Truth" Business Knowledge:** Answering business questions needs deep, specific knowledge drawn from many sources (e.g., table metadata, company documents, internal messages) that are often outdated, contradictory, or superseded, forcing the agent to determine the most authoritative information.
- **Lack of Verifiable Tests:** Unlike coding agents that can use deterministic, verifiable tests to iteratively refine code, data agents have no corresponding test because the "specification" is just the high-level user query without a notion of the expected correct answer. Moreover, the queries may not always be answerable because of incompleteness in data, and it is important for data agents to be able to identify such cases and surface it back to users.

“

We have two dashboards for Product X revenue — one is the KPI Dashboard, the other is the Active Customer Dashboard. They show different numbers for the same time period. Can you figure out why they disagree, which one is correct, and verify the math for specific customers — for example, Customer A’s invocations dropped by $\sim Xk$ but revenue only changed by $\sim \$Y$, which doesn’t seem right.



Resolved. The gap is structural, not a bug — and Customer A’s math closes to the dollar: $Xk \cdot \$p \approx \Y ✓

Figure 2: An example trajectory showing how Genie solves a complex user query across different phases: parallel multi-agent asset discovery, data investigation (SQL extraction, comparative analysis, root-cause investigation), self-correction and reconciliation, and final verification.

Key Technical Advances

Figure 3 shows some of the key technical innovations in Genie that enable it to perform significantly better than generic coding agents, namely: i) Specialized Knowledge Search, ii) Parallel Thinking, and iii) Multi-LLM. Specialized knowledge search uses semantic contextual data to ground the asset discovery sub-agents and significantly improve the search quality. Parallel thinking allows the agent to sample multiple different trajectories and then aggregate the findings across trajectories to compute the final answer. Finally, Multi-LLM allows the agent to use different LLMs for each of the different sub-agents together with their optimized prompts to further improve the overall accuracy and latency.

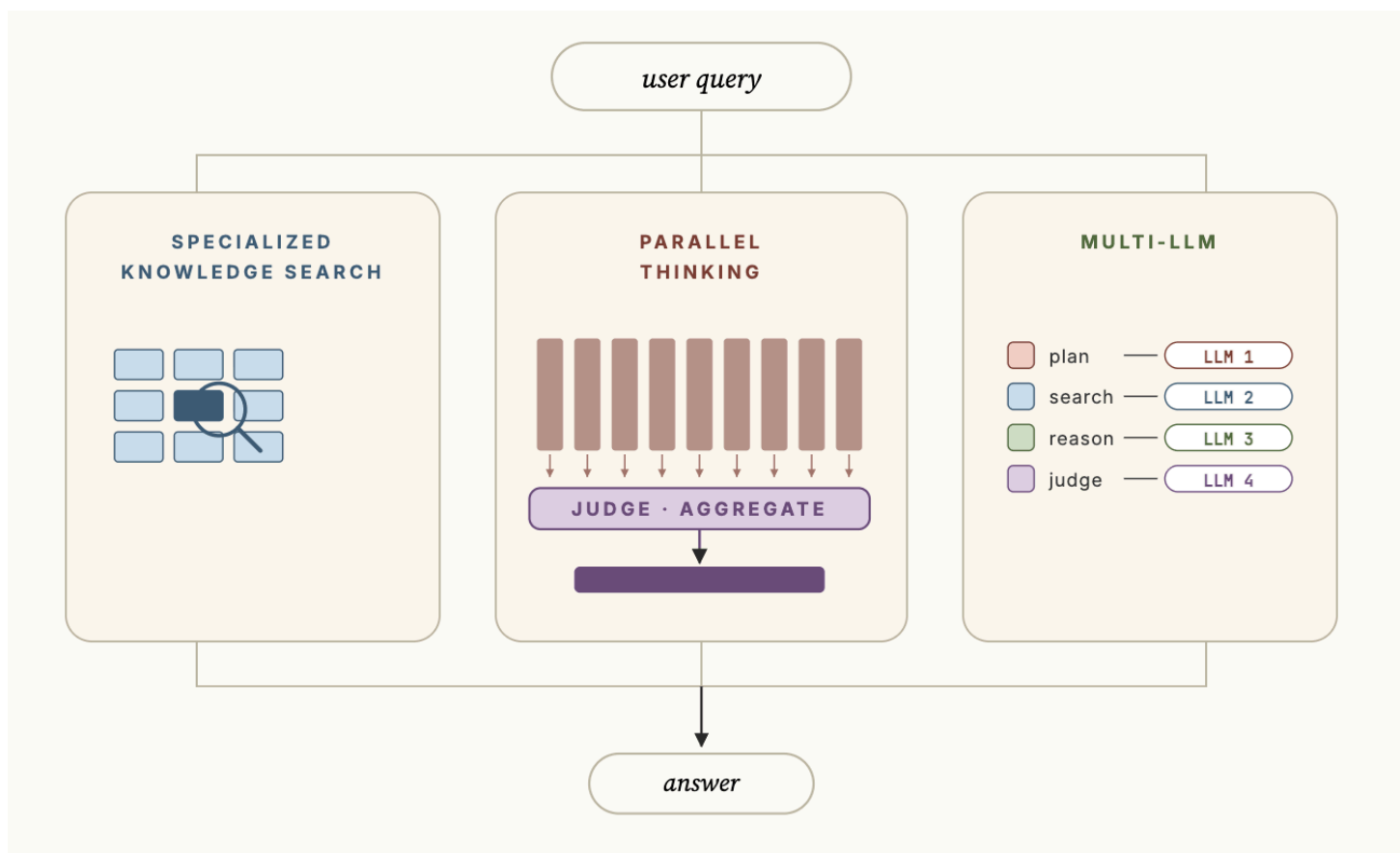


Figure 3: The key technical advances in Genie: i) Specialized Knowledge Search, ii) Parallel Thinking, and iii) Multi-LLM that allow for significant improvements in accuracy and latency.

Specialized Knowledge Search

Genie uses the existing data assets such as workspace tables, notebooks, dashboards, documents, and files to derive a rich semantic enterprise context and then uses this context to construct a search index. It uses multiple search indices in parallel together with rich metadata signals to efficiently discover most relevant assets for a user query. Figure 4 demonstrates how leveraging the specialized knowledge search helps Genie improve table search performance by up to 40% on our table discovery benchmarks.

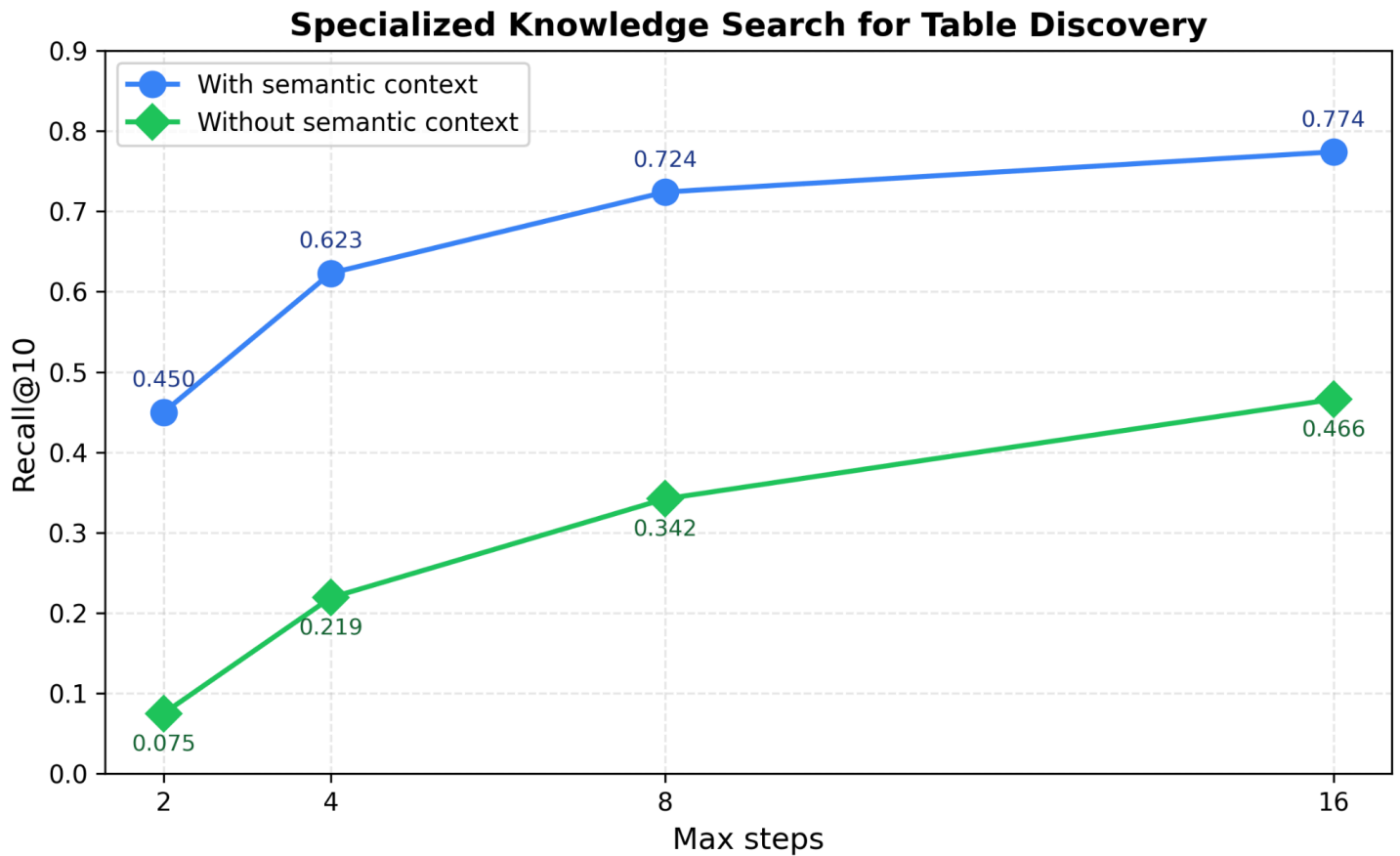


Figure 4: Comparison of Specialized Knowledge Search for Table Search performance.

Parallel Thinking

Unlike software engineering tasks, where coding agents can first write tests to verify the desired functionality and then iterate on code generation until the tests pass, the open-ended data queries don't have such corresponding unit tests. In the absence of tests, it becomes challenging for data agents to know if the generated answer is correct or needs more refinement. To address this challenge, we leverage parallel thinking by sampling multiple trajectories and aggregating relevant information across the trajectories to compute the final answer. Figure 5 shows how parallel thinking can significantly improve the answer accuracy, although with some additional latency and token costs. Furthermore, as shown in Figure 1, combining Multi-LLM and further optimizations can further significantly reduce costs and latency.

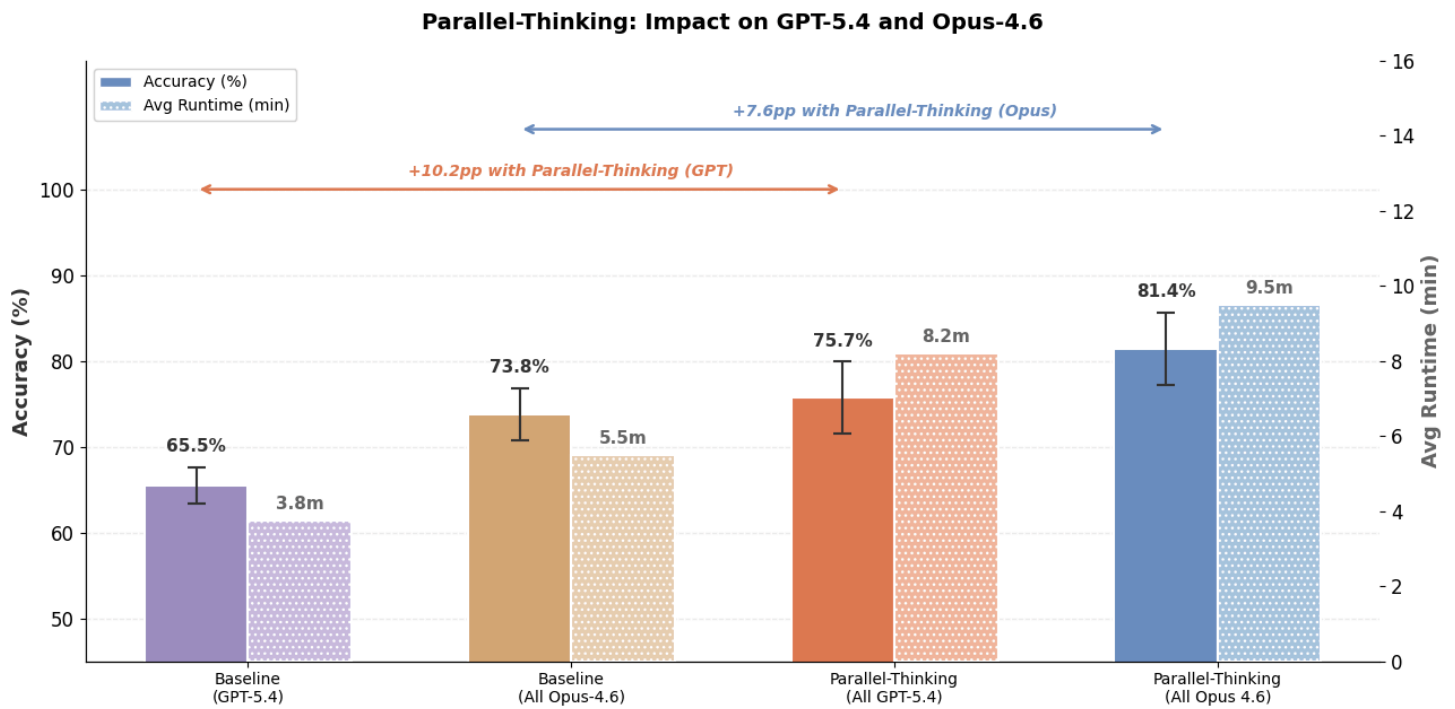


Figure 5: Adding parallel thinking improves overall performance across both GPT-5.4 and Opus-4.6.

Multi-LLM

One of the key technical advances in Genie is the ability to leverage different LLMs for different sub-agents as we observe different LLMs are good at complementary capabilities. For example, it can use a different LLM for the planning stage, a different LLM for various search sub-agents, a different one for code generation and judges. With the Databricks platform, it is seamless to try out any of the frontier models (including Opus, GPT, and Gemini), open-source models, as well as custom trained models. In addition to accuracy, we also observe that different LLMs result in very different latency and cost characteristics. Figure 6 shows how different LLMs perform on table search tasks and how the corresponding accuracy and cost can be further optimized using methods like [GEPA](#).

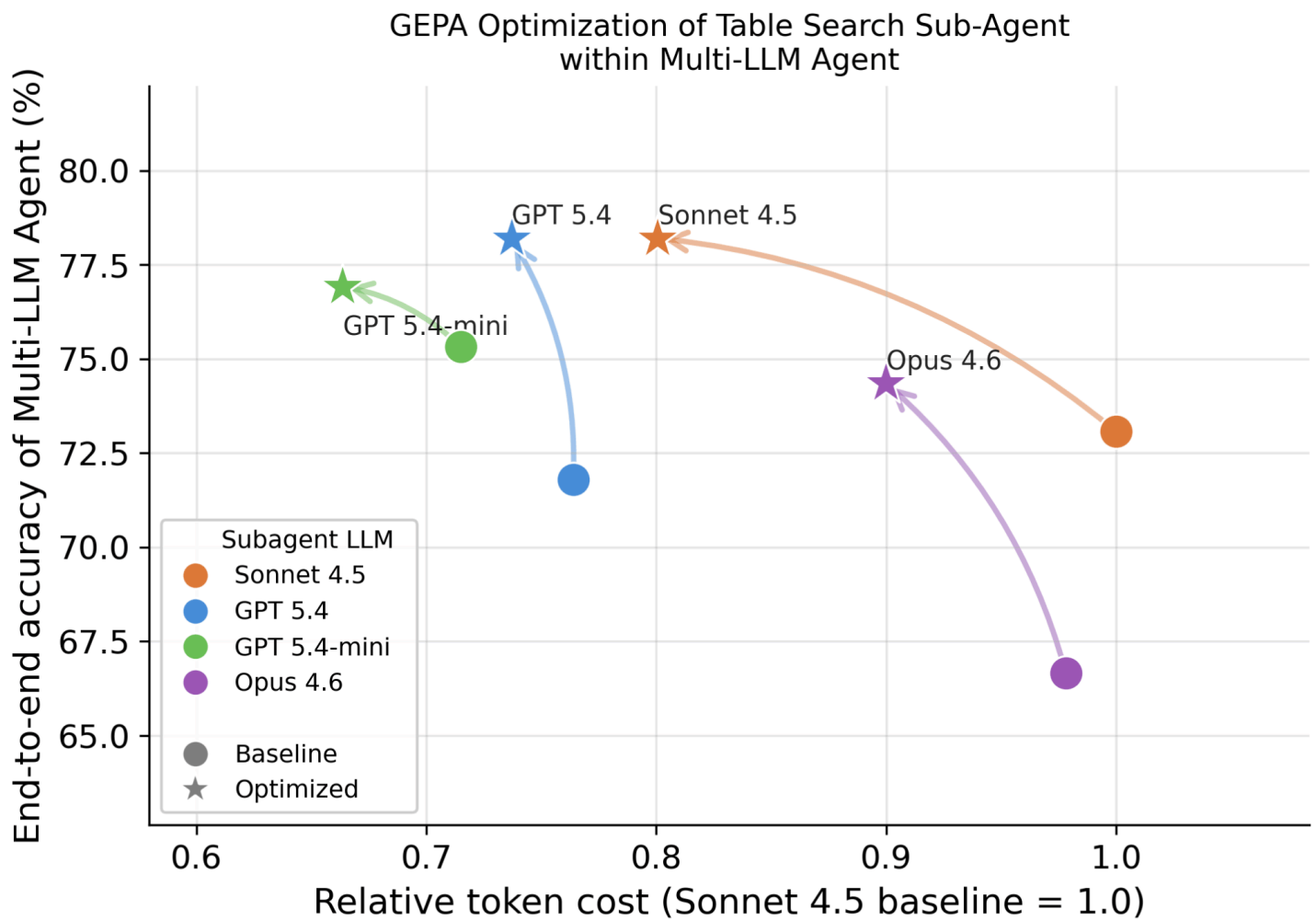


Figure 6: Optimizing the accuracy and cost for different LLMs for Table Search using GEPA.

Conclusion

While coding and data analysis share many conceptual similarities, the dynamic nature of enterprise data systems create some unique challenges. Data agents need to efficiently discover the right assets from a large enterprise context, determine “truth” in an ambiguous environment and write efficient code and queries to correctly answer user's questions. We developed several novel approaches to solve these problems such as specialized knowledge search to leverage rich semantic information and multiple metadata signals, Multi-LLM to leverage different LLMs with optimized prompts using GEPA, and parallel thinking to further improve the overall accuracy. Adding these approaches to Genie helps it perform significantly better than leading coding agents on the benchmark tasks. There are still a lot of challenging open-ended questions left to explore, and it has never been a more exciting time to explore research in this area of building state-of-the-art data agents for enterprises.